03

AULA 03 · Sexta, 14/08

Apache e Nginx

Os servidores web mais usados do planeta

Módulo · Arquitetura de Serviços e Dados

Prof.: Everton J Benedicto

ada

Cronograma das Aulas

01
Seg 10/08
Introdução ao Módulo e Docker

02
Qua 13/08
Arquitetura de Servidores

03
Sex 14/08
Apache e Nginx

04
Seg 17/08
Proxy Reverso e DNS

05
Qua 19/08
CI/CD e Ciclo de Vida

06
Sex 21/08
Cloud Computing

07
Seg 24/08
Arquitetura de Dados e Data Warehouse

08
Qua 24/08
Data Lake

09
Qua 25/08
Concluir atividades e preencher rubricas

COMBINADO DA TURMA

Por favor!

Bora participar!



Câmera ligada + mão levantada
= aula top!

INTERVALO · 16:30 BRT

Combinados da nossa aula

1. Câmeras ligadas.
2. Entrem no horário: início e intervalo.
3. Participem da aula.
4. Tirem dúvidas.
5. Levantem a mão para tirar dúvidas.

REVIEW!

Recapitulando a Aula 02

- ✓ Monolítico é simples no início; microserviços são flexíveis, porém mais complexos.
- ✓ Cada tipo de servidor tem uma função específica na arquitetura.
- ✓ Escalabilidade vertical (mais poder) vs horizontal (mais máquinas).
- ✓ Servidores de aplicação gerenciam transações, segurança e pools de conexão.

SERVIDOR WEB

Apache HTTP Server — O Veterano



1. **Criado em 1995:** um dos mais populares do mundo
2. **Open source:** multiplataforma e altamente customizável
3. **Modular:** adicione PHP, Python e SSL com módulos
4. **Por conexão:** usa um processo ou thread para cada requisição

SERVIDOR WEB

Apache — Vantagens e Desvantagens



Vantagens

- **Amplo ecossistema de módulos:** centenas de módulos oficiais e de terceiros
- **Alta compatibilidade:** roda WordPress, Joomla, Drupal e apps PHP sem ajustes
- **Configuração distribuída:** o .htaccess permite regras por diretório, sem reiniciar
- **Comunidade enorme:** documentação sólida e suporte fácil de encontrar

Desvantagens

- **Consumo de memória alto:** cada conexão gera processo ou thread próprio
- **Escalabilidade limitada:** perde para arquiteturas event-driven como o Nginx
- **Configuração mais complexa:** muitos arquivos e diretivas para dominar
- **.htaccess custa desempenho:** lido a cada requisição em todos os diretórios

NA PRÁTICA

Apache — Casos de Uso

1

Hospedagem compartilhada

HostGator, Locaweb e afins: cada cliente configura seu próprio site sem tocar no servidor.

2

Sites PHP tradicionais

WordPress, Moodle e Joomla rodam com mod_php ou PHP-FPM praticamente sem ajustes.

3

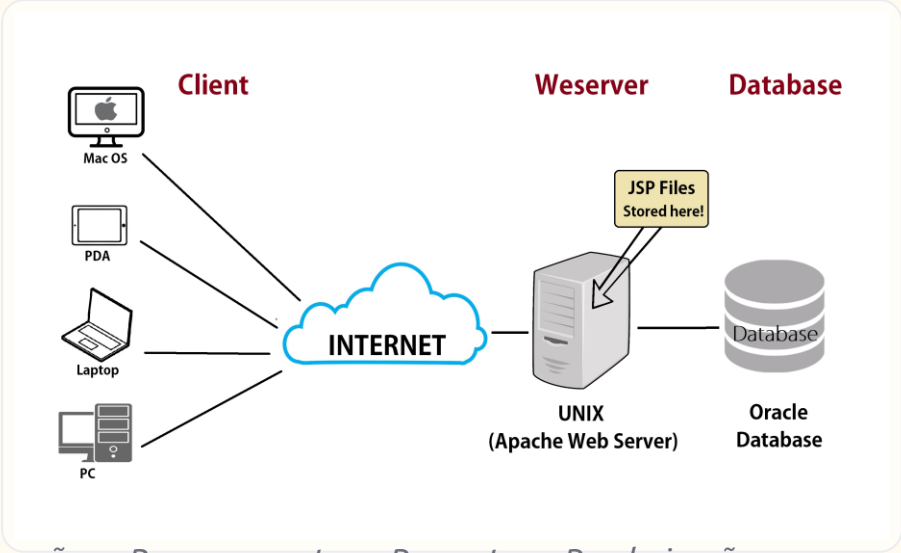
Sistemas legados

Dependem do .htaccess para redirecionamentos, reescrita de URLs e regras de acesso.

Regra prática: escolha o Apache quando compatibilidade e flexibilidade de configuração importam mais que o volume de conexões.

COMO FUNCIONA

O Funcionamento de um Servidor Web



Recepção → Processamento → Resposta → Renderização no navegador.

Fonte: informationq.com

MÃO NA MASSA

Apache com Docker

Suba um Apache sem instalar nada além do Docker:

bash

```
# Baixar a imagem oficial (httpd)
docker pull httpd

# Rodar o Apache na porta 80
docker run -d --name meu-apache -p 80:80 httpd

# Verificar
docker ps

# Abra http://localhost:80  -> 'It works!'
# Desafio: altere a página inicial!
```

MÃO NA MASSA

Resolução do Desafio — Trocar a Página Inicial

Com o container do slide anterior no ar, substitua o index.html do Apache:

powershell

```
# 1. Confirma que o container esta em execucao
docker ps --quiet --filter "name=^/meu-apache$"

# 2. Cria backup da pagina atual (dentro do container)
docker exec meu-apache sh -c "if [ -f /usr/local/apache2/htdocs/index.html ]; then \
  cp /usr/local/apache2/htdocs/index.html /usr/local/apache2/htdocs/index.html.backup; fi"

# 3. Copia o novo HTML da sua maquina para o container
docker cp ".\site\index.html" "meu-apache:/usr/local/apache2/htdocs/index.html"

# 4. Confirma que o arquivo foi copiado
docker exec meu-apache ls -l /usr/local/apache2/htdocs/index.html

# Recarregue http://localhost:80  ->  sua propria pagina no ar!
```

SERVIDOR WEB

Nginx — O Campeão de Performance



1. **Criado em 2002 por Igor Sysoev:** resolveu o C10k, o limite de 10 mil conexões que derrubava os servidores da época
2. **Assíncrono:** como um garçom que anota o pedido e já atende outra mesa, não fica parado esperando; gasta pouca RAM
3. **Poucos workers:** cerca de 1 por núcleo de CPU, cada um alternando entre milhares de conexões
4. **Versátil:** servidor web, proxy reverso para Node, Java ou Python, balanceador de carga e cache

SERVIDOR WEB

Nginx — Vantagens e Desvantagens

Vantagens

- **Desempenho em estático:** entrega HTML, CSS, JS e imagens com uso mínimo de memória
- **Proxy reverso e balanceamento:** distribui carga entre várias aplicações e servidores
- **Cache embutido:** guarda respostas e alivia o backend sem ferramenta extra
- **Configuração direta:** server blocks claros e centralizados em poucos arquivos

Desvantagens

- **Sem .htaccess:** config só com o admin; exige reload a cada mudança
- **Menos módulos:** ecossistema menor e módulos costumam exigir recompilação
- **Não processa PHP:** precisa delegar a um PHP-FPM externo
- **Curva inicial:** eventos e upstreams mudam a lógica de quem vem do Apache

NA PRÁTICA

Nginx — Casos de Uso



1

Portais e streaming

Alto volume de acessos simultâneos e entrega de vídeo e arquivos estáticos com pouca memória.

2

Gateway de API

Porta de entrada dos microsserviços: roteia, balanceia carga, aplica TLS e limita requisições.

3

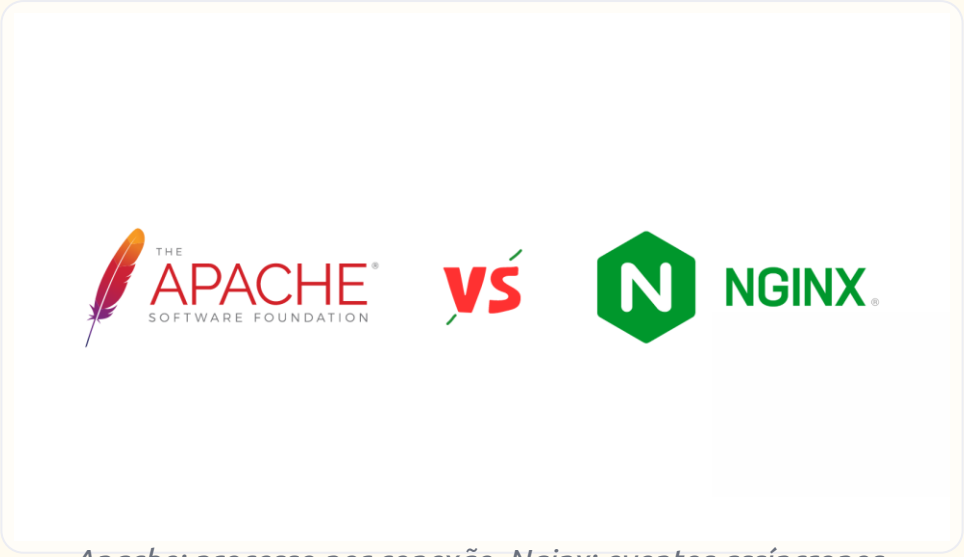
Frontend moderno

Serve o build de React ou Angular e encaminha /api para o backend, evitando problemas de CORS.

Quem usa: Netflix, Airbnb, GitHub, Dropbox e Instagram — todos com tráfego na casa dos milhões de requisições.

COMPARAÇÃO

Apache vs Nginx



Apache: processo por conexão. Nginx: eventos assíncronos.

Fonte: insightdials.com

COMPARAÇÃO DETALHADA

Apache vs Nginx — Ponto a Ponto

APACHE HTTP SERVER PROJECT

- **Desempenho:** integra Java (mod_jk/Tomcat) e .NET via mod_proxy; mais lento em estáticos
- **Configuração:** .htaccess por diretório: muita flexibilidade, custo de desempenho
- **Casos de uso:** hospedagem compartilhada e apps Java legadas em Tomcat

NGINX

- **Desempenho:** muito mais rápido em HTML, CSS e imagens; Java e .NET sempre via proxy
- **Configuração:** centralizada no servidor: mais velocidade e segurança
- **Casos de uso:** altíssimo tráfego, proxy reverso, load balancer e API gateway

Abordagem híbrida

Usuário → **Nginx · estáticos + proxy** → **Tomcat / Kestrel · Java e .NET**

MÃO NA MASSA

Nginx com Docker Compose

Um servidor Nginx com página customizada via Compose:

yml

```yml
# docker-compose.yml
name: aula-03-nginx
services:
  web:
    image: nginx:1.28-alpine
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html
    restart: unless-stopped
    healthcheck:
      test: ["CMD-SHELL", "wget --quiet --spider http://localhost/ || exit 1"]
      interval: 10s
      timeout: 3s
      retries: 3
      start_period: 5s
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"
```

RESUMO

Recapitulando a Aula

- ✓ Apache: veterano e modular, uma thread/processo por requisição — ótimo para flexibilidade.
- ✓ Nginx: assíncrono por eventos — ótimo para alta concorrência e proxy reverso.
- ✓ Podem trabalhar juntos: Nginx na frente (proxy) + Apache/Tomcat atrás (app).
- ✓ Docker facilita instalar e testar qualquer servidor web.